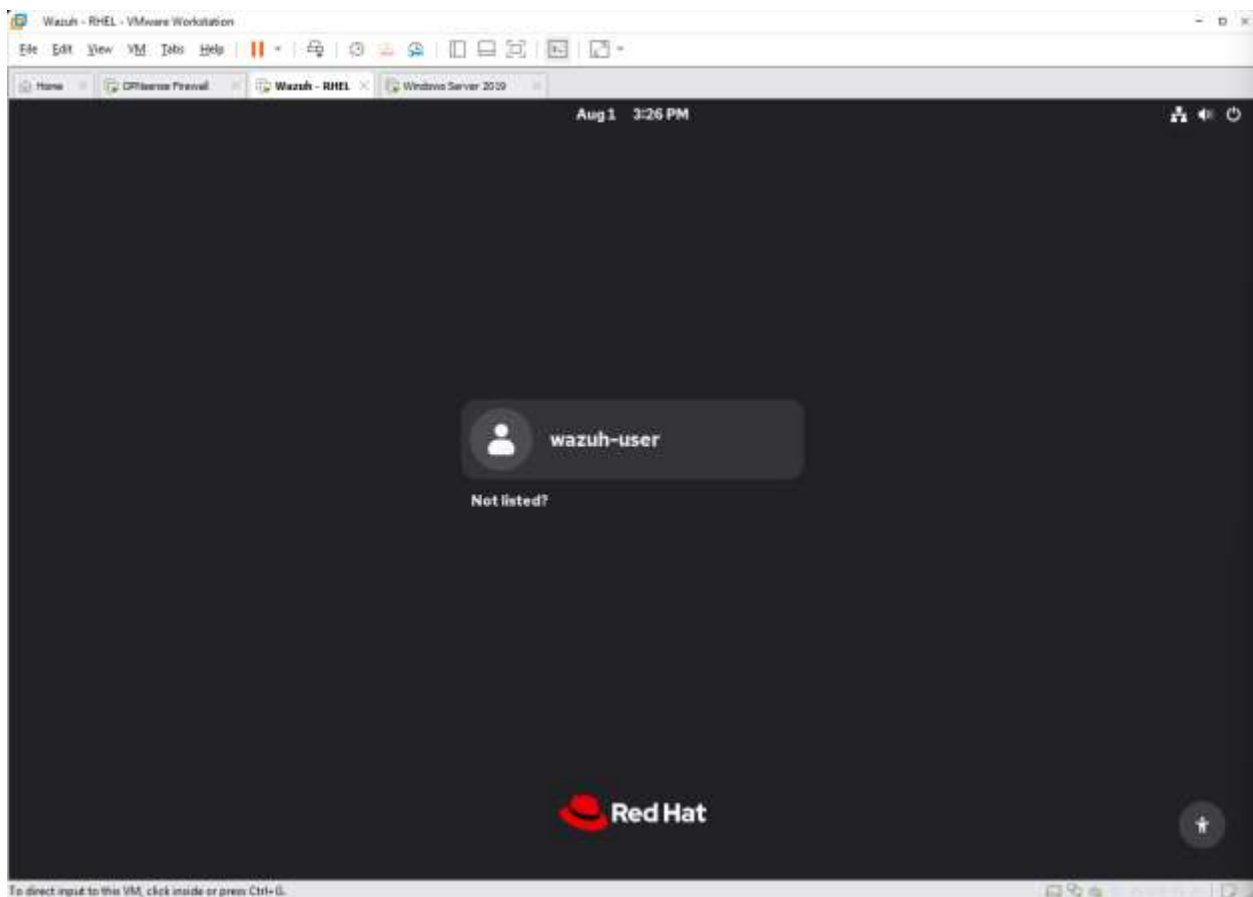


SOC Home Lab / Phase 2

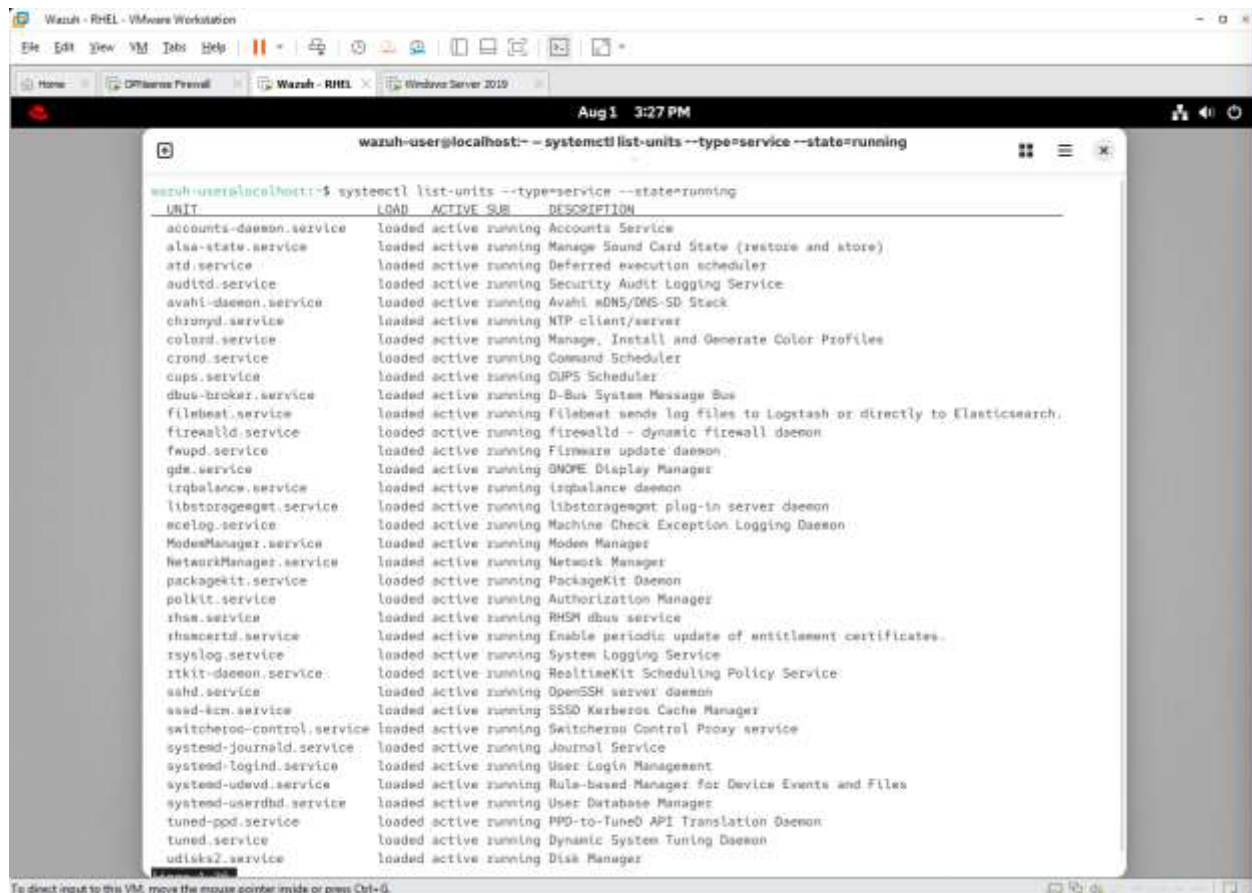
Author: Andy Hernandez

Objective: The objective of phase two is to deploy Wazuh agents to the following systems: Windows Domain Controller (Windows Server 2019 VM) and Windows 10 Workstation. I am using an open-source SIEM (Wazuh) to gain hands-on experience with centralized log aggregation, endpoint telemetry, and rule management.

Deploying Wazuh Agents: Wazuh comes with a few ways to deploy agents but the one I found easiest was using the web UI to generate a command you can copy and paste into your virtual machine's terminal. I first boot up my RHEL 10 VM and verify the Wazuh services are running properly before I head to the web UI.



Here I listed all active running services on my RHEL system and proceeded to search for any services with the name “*Wazuh*”.



```
wazuh-user@localhost:~$ systemctl list-units --type=service --state=running
```

UNIT	LOAD	ACTIVE	STATE	DESCRIPTION
accounts-daemon.service	loaded	active	running	Accounts Service
alsa-state.service	loaded	active	running	Manage Sound Card State (restore and store)
atd.service	loaded	active	running	Deferred execution scheduler
audit.service	loaded	active	running	Security Audit Logging Service
avahi-daemon.service	loaded	active	running	Avahi mDNS/DNS-SD Stack
chronyd.service	loaded	active	running	NTP client/server
colord.service	loaded	active	running	Manage, Install and Generate Color Profiles
crond.service	loaded	active	running	Command Scheduler
cups.service	loaded	active	running	CUPS Scheduler
dbus-broker.service	loaded	active	running	D-Bus System Message Bus
filebeat.service	loaded	active	running	Filebeat sends log files to Logstash or directly to Elasticsearch.
firewalld.service	loaded	active	running	firewalld - dynamic firewall daemon
fwupd.service	loaded	active	running	Firmware update daemon
gdm.service	loaded	active	running	GDM: Display Manager
irqbalance.service	loaded	active	running	irqbalance daemon
libstoragesmgt.service	loaded	active	running	libstoragesmgt plug-in server daemon
mcelog.service	loaded	active	running	Machine Check Exception Logging Daemon
ModemManager.service	loaded	active	running	Modem Manager
NetworkManager.service	loaded	active	running	Network Manager
packagekit.service	loaded	active	running	PackageKit Daemon
polkit.service	loaded	active	running	Authorization Manager
rshim.service	loaded	active	running	RHSM dbus service
rhsmcertd.service	loaded	active	running	Enable periodic update of entitlement certificates.
rsyslog.service	loaded	active	running	System Logging Service
rtkit-daemon.service	loaded	active	running	RealtimeKit Scheduling Policy Service
sasl.service	loaded	active	running	OpenSSH server daemon
sssd-kcm.service	loaded	active	running	SSSD Kerberos Cache Manager
switcheroo-control.service	loaded	active	running	Switcheroo Control Proxy service
systemd-journald.service	loaded	active	running	Journal Service
systemd-logind.service	loaded	active	running	User Login Management
systemd-udev.service	loaded	active	running	Rule-based Manager for Device Events and Files
systemd-userdbd.service	loaded	active	running	User Database Manager
tuned-pod.service	loaded	active	running	PPD-to-Tuned API Translation Daemon
tuned.service	loaded	active	running	Dynamic System Tuning Daemon
udisks2.service	loaded	active	running	Disk Manager

It's important to note that Wazuh relies on three core services working together to have a fully functional SIEM. The three core services are as follows:

- **Wazuh-Dashboard.service** – The web-based user interface. It queries the Indexer to display visual dashboards, threat analytics, system alerts, and platform management tools.
- **Wazuh-Indexer.service** – This service is a high-performance search and storage engine. It indexes processed security alerts from the manager.
- **Wazuh-Manager.service** – The core processing engine ("the brain"). It collects raw telemetry from agents, parses it through decoders and detection rules, triggers alerts, and forwards the processed events to the Indexer.

Here you can see that all the required services are running as expected but if you would like to inspect the services in more detail you can use the command “`systemctl status <service-name>`”.

```
wazuh-user@localhost:~ -- systemctl list-units --type=service --state=running

gdm.service                                loaded active running GNOME Display Manager
irqbalance.service                        loaded active running irqbalance daemon
libstoragemgmt.service                   loaded active running libstoragemgmt plug-in server daemon
acelog.service                           loaded active running Machine Check Exception Logging Daemon
ModemManager.service                     loaded active running Modem Manager
NetworkManager.service                  loaded active running Network Manager
packagekit.service                       loaded active running PackageKit Daemon
polkit.service                           loaded active running Authorization Manager
rhnssd.service                           loaded active running RHEM dbus service
rhsmcertd.service                        loaded active running Enable periodic update of entitlement certificates.
rsyslog.service                          loaded active running System Logging Service
rtkit-daemon.service                     loaded active running RealtimeKit Scheduling Policy Service
sshd.service                             loaded active running OpenSSH server daemon
sssd-kcm.service                         loaded active running SSSD Kerberos Cache Manager
switcheroo-control.service               loaded active running Switcheroo Control Proxy service
systemd-journald.service                  loaded active running Journal Service
systemd-logind.service                   loaded active running User Login Management
systemd-udev.service                     loaded active running Rule-based Manager for Device Events and Files
systemd-userdbd.service                  loaded active running User Database Manager
tuned-pod.service                        loaded active running PPD-to-Tuned API Translation Daemon
tuned.service                            loaded active running Dynamic System Tuning Daemon
udisks2.service                          loaded active running Disk Manager
upower.service                           loaded active running Daemon for power management
user@1000.service                        loaded active running User Manager for UID 1000
vgauthd.service                          loaded active running VGAuthService for open-vn-tools
vmtolad.service                          loaded active running Service for virtual machines hosted on VMware

wazuh-dashboard.service                  loaded active running wazuh-dashboard
wazuh-indexer.service                    loaded active running wazuh-indexer
wazuh-manager.service                    loaded active running Wazuh manager
wpa_supplicant.service                   loaded active running WPA supplicant

Legend: LOAD    -> Reflects whether the unit definition was properly loaded.
        ACTIVE -> The high-level unit activation state, i.e. generalization of SUB.
        SUB    -> The low-level unit activation state, values depend on unit type.

43 loaded units listed.
lines 15-50/50 (END)
```

Here is a quick visual representation of the command mentioned above:

```
root@localhost:~# sudo -i

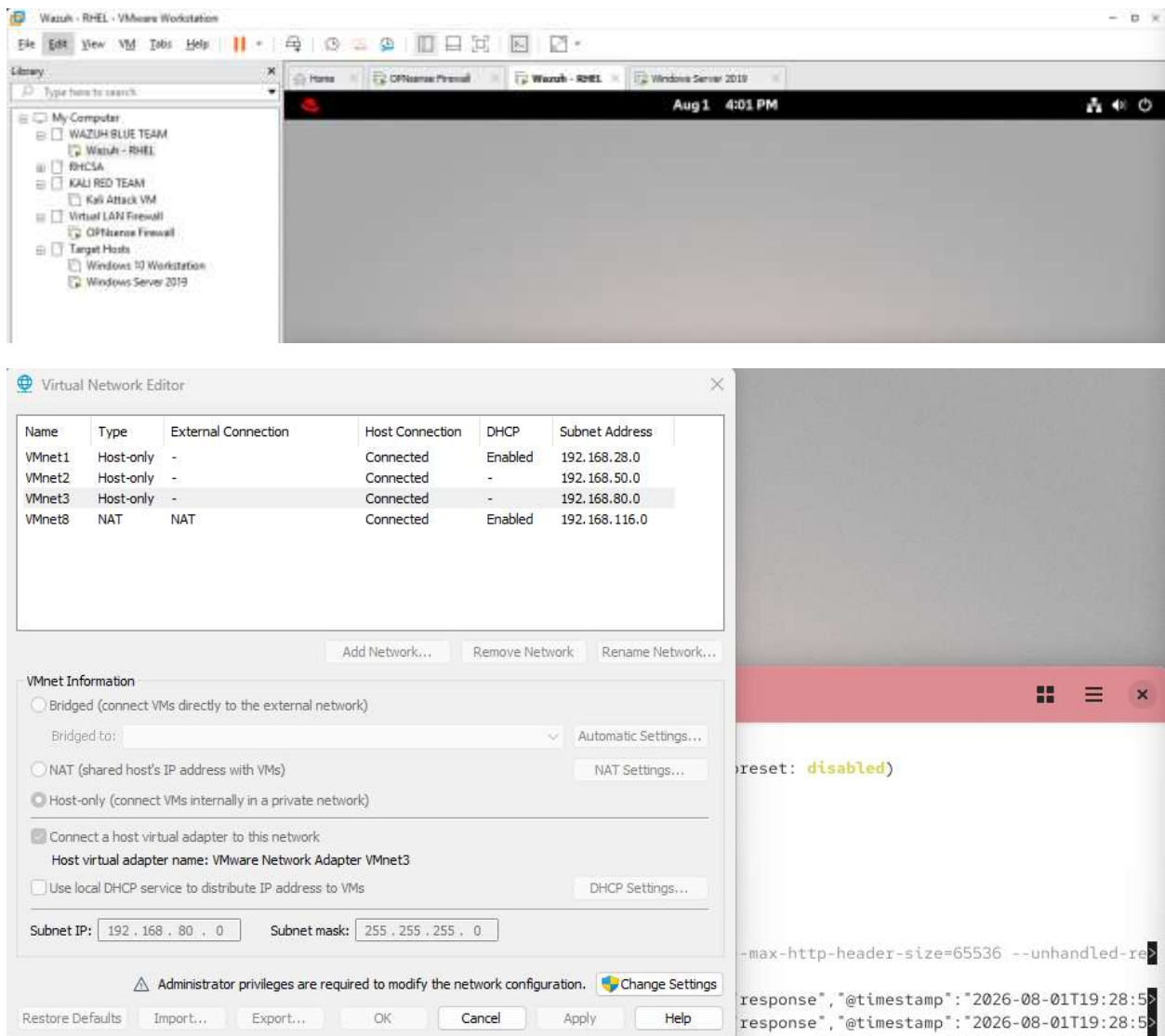
root@localhost:~# systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-01 15:29:39 EDT; 16min ago
   Invocation: 8814f46bea244855dbbbs223e91ca1f94
   Main PID: 1277 (node)
     Tasks: 11 (limit: 974512)
    Memory: 289.1M (peak: 367.4M)
       CPU: 5.576s
    CGroup: /system.slice/wazuh-dashboard.service
           └─1277 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --amandlad-seject

Aug 01 15:28:59 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:28:59Z"}
Aug 01 15:28:59 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:28:59Z"}
Aug 01 15:28:59 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:28:59Z"}
Aug 01 15:28:59 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:28:59Z"}
Aug 01 15:28:59 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:28:59Z"}
Aug 01 15:29:01 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:29:01Z"}
Aug 01 15:29:01 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:29:01Z"}
Aug 01 15:29:01 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:29:01Z"}
Aug 01 15:29:01 localhost.localdomain opensearch-dashboards[1277]: {"type": "response", "timestamp": "2026-08-01T19:29:01Z"}
lines 1-21/21 (END)
```

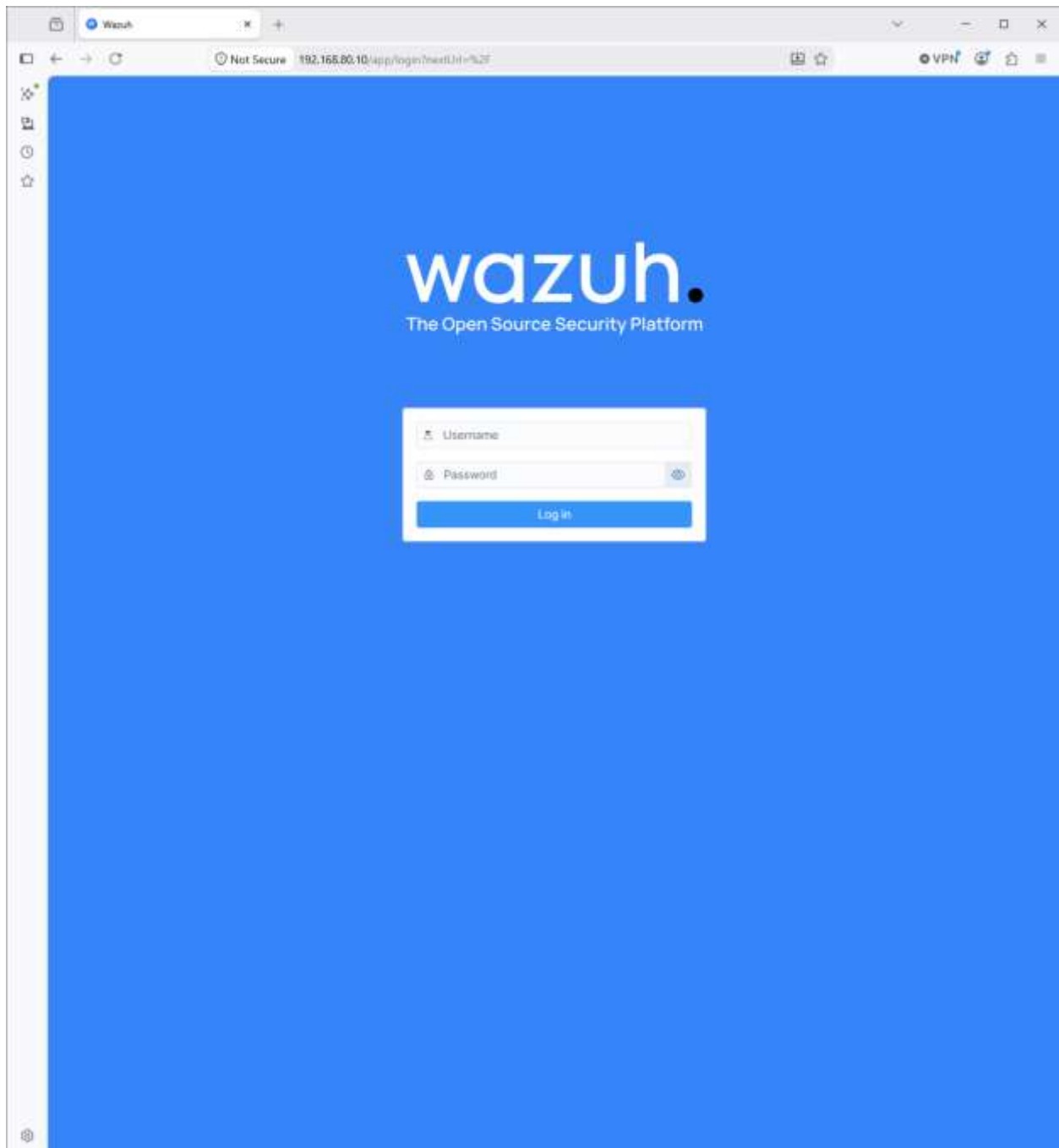
Now that all the initial checks have been completed it is time to use the web UI to deploy agents to the following systems: Windows Server 2019 & Windows 10 Workstation.

The web UI is located at <https://<Wazuh-Machine-IP>:443>

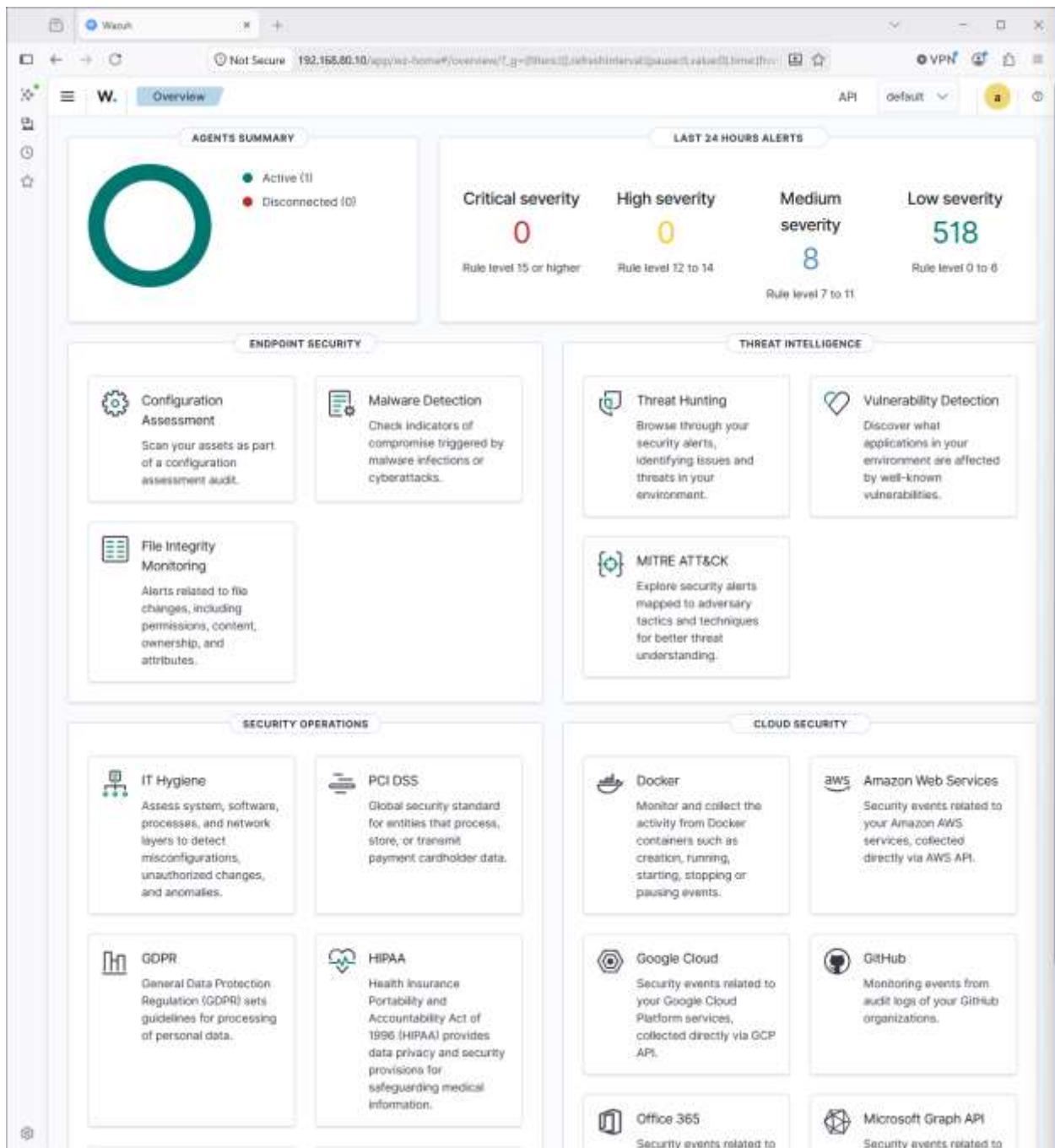
Before connecting ensure the setting “**Connect a host virtual adapter to this network**” is enabled on VMware Workstation or you may need to create a port forward rule for the WAN interface on the OPNsense firewall. This enabled configuration will allow direct communication between your host machine and the machine hosting your web UI. You can find the settings in the “**edit**” tab and selecting “**virtual network editor**”. Here is a visual representation:



Keep in mind that you will replace the field “Wazuh-Machine-IP” with the IPv4 address of the machine you installed the core services on. Here is the webpage you should see before logging in.

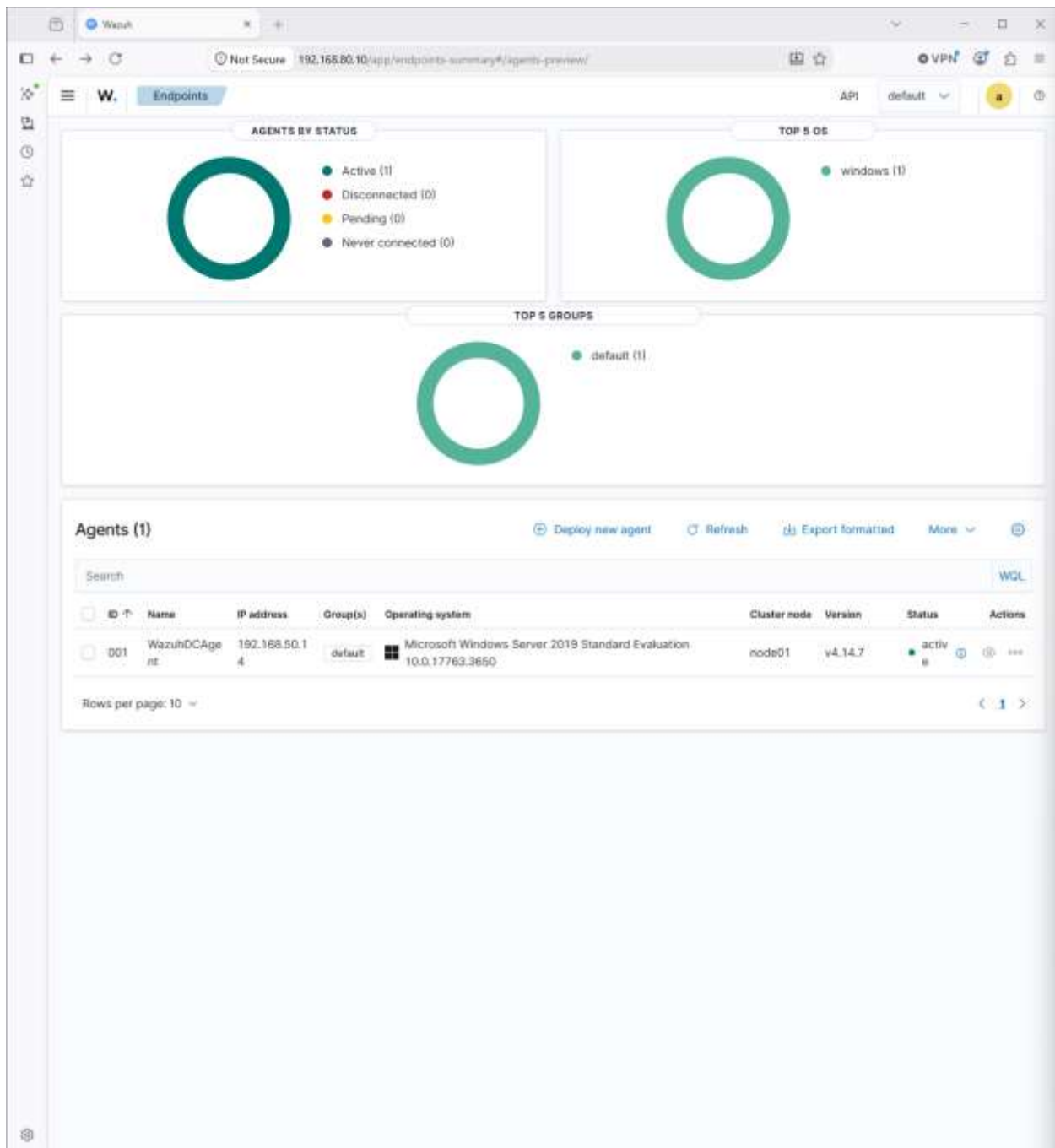


From here I logged in using the administrator credentials generated for me during the phase one setup. Here is the initial dashboard you will see once logged in:



You may notice that I have one agent active in my agent summary. After your initial setup (installing Wazuh and core services), that will not be there, but you will have an option to deploy a new agent which is very straightforward.

I am going to click on the *active agents* section on my dashboard and *deploy new agent* links.



You will then be presented with this page where you will fill out some information about the system you are deploying the agent to.

W. Endpoints Deploy new agent API default

Deploy new agent

1 Select the package to download and install on your system:

 **LINUX**

☐ RPM amd64 ☐ RPM aarch64
☐ DEB amd64 ☐ DEB ppc64

 **WINDOWS**

☐ MSI 32/64 bits

 **macOS**

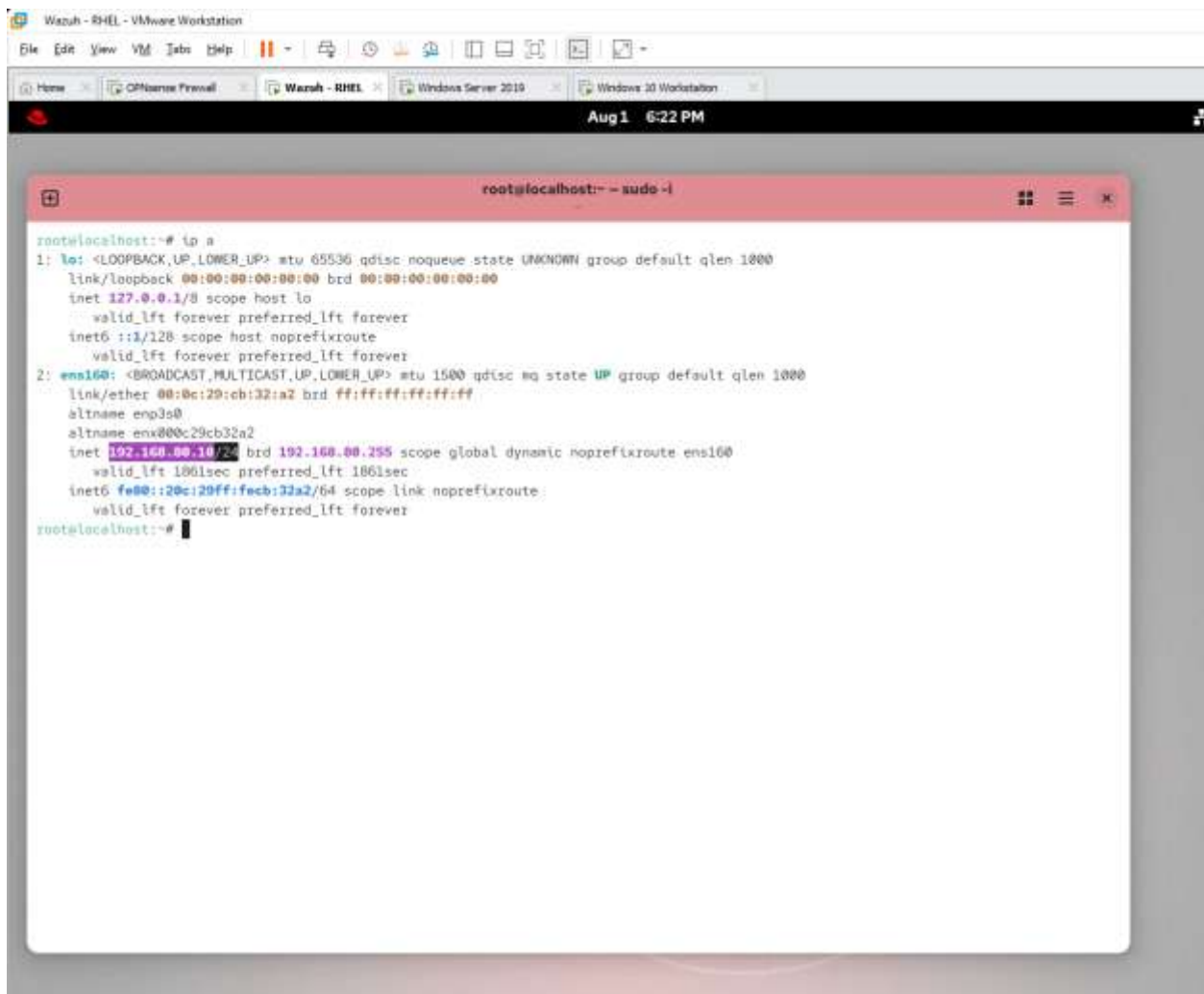
☐ Intel ☐ Apple silicon

I will be showing my initial deployment for Windows 10 Workstation as an example.

First I will begin by selecting the operating system I am deploying the agent to. This matters because it determines the packages that will be installed on your system to ensure the agent is functioning properly.

I will then enter the server address or in other words the IPv4 address of the central server which is the RHEL 10 virtual machine.

I will first validate the IP address of the RHEL machine to ensure I am entering the correct information. I will enter the command “*ip a*” into the terminal.



```
root@localhost:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:cb:32:a2 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    altnam ens00c29cb32a2
    inet 192.168.80.10/24 brd 192.168.00.255 scope global dynamic noprefixroute ens160
        valid_lft 1861sec preferred_lft 1861sec
    inet6 fe80::20c:29ff:feeb:32a2/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
root@localhost:~#
```

As you can see the IPv4 address I will be using is 192.168.80.10, keep in mind the CIDR notation at the end is not required.

Switching back to the Wazuh web UI we will now enter the address 192.168.80.10 as well as assign an agent name in step three. I will be entering the name WindowsWorkstation1Agent:

Wazuh

Rules | Firewall | OPNsense.Bur | +

Not Secure 192.168.80.10/app/endpoints-summary#/agents-preview/deploy

W. Endpoints Deploy new agent API default

Deploy new agent

1 Select the package to download and install on your system:

LINUX

☐ RPM amd64 ☐ RPM aarch64

☐ DEB amd64 ☐ DEB aarch64

WINDOWS

☒ MSI 32/64 bits

macOS

☐ Intel

☐ Apple silicon

For additional systems and architectures, please check our [documentation](#).

2 Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address

192.168.80.10

☒ Remember server address

3 Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name:

WindowsWorkstation1Agent

The agent name must be unique. It can't be changed once the agent has been enrolled.

Select one or more existing groups:

Default

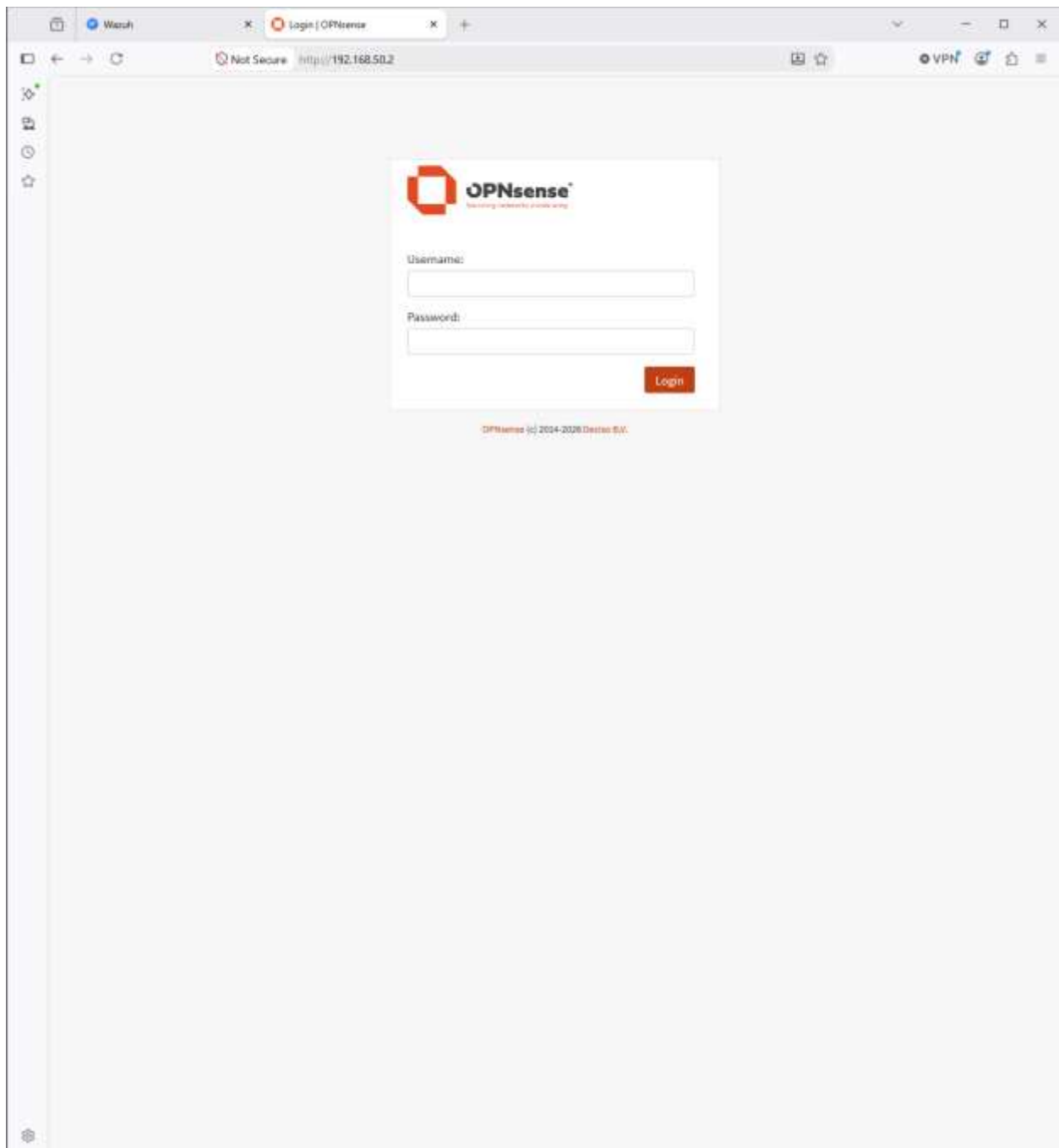
4 Run the following commands to download and install the agent:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.7-1.msi -OutFile $env:temp\wazuh-agent.msi && .\wazuh-agent.msi /S /q /q WAZUH_MANAGER=192.168.80.10 WAZUH_AGENT_NAME=WindowsWorkstation1Agent
```

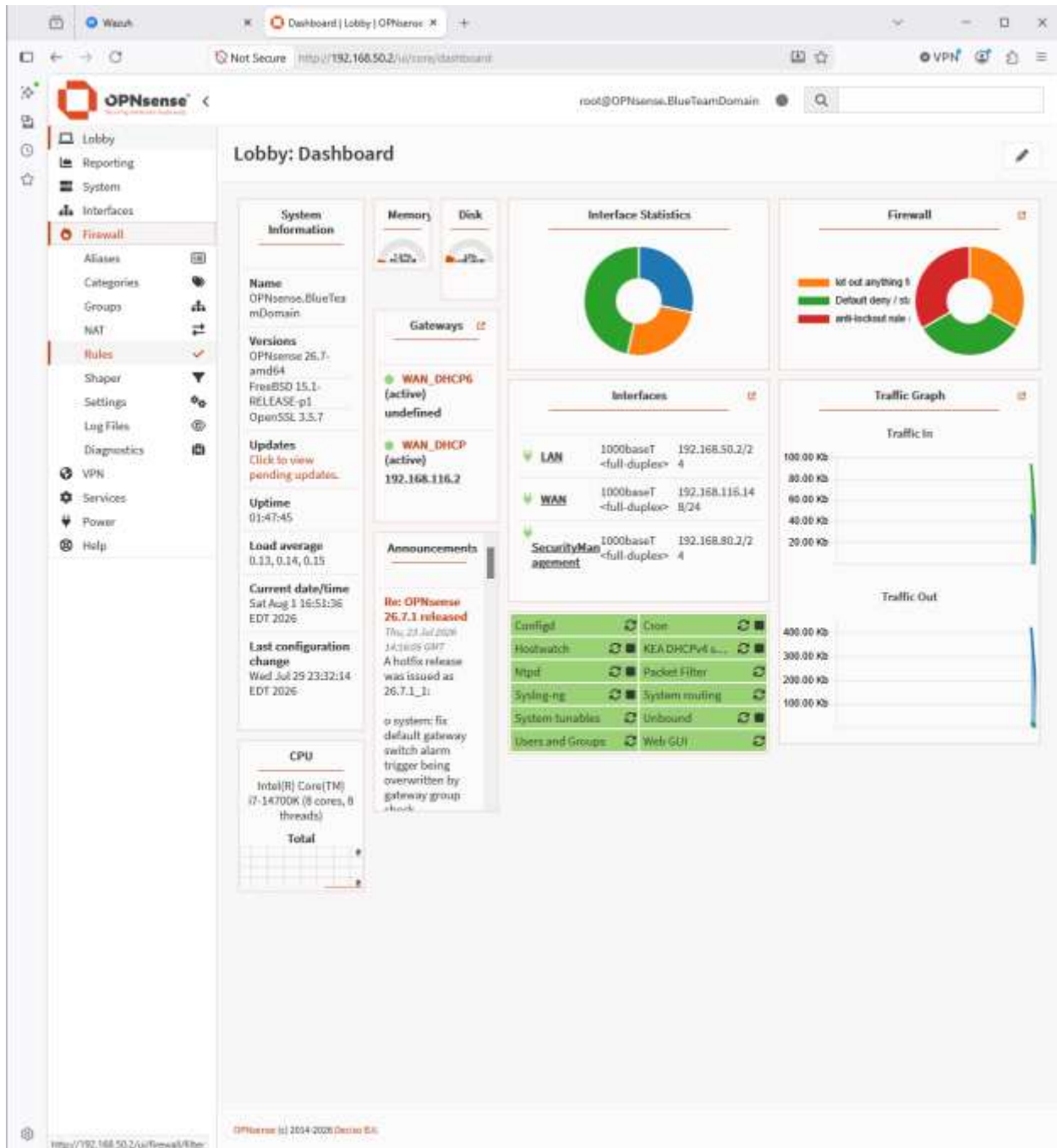
Activate Windows
Go to Settings to activate Windows.

You may leave the optional step as default and below you will see a command that you will be entering in the PowerShell terminal, but I will save that command for later as we need to adjust the firewall rules in OPNsense to allow the agents to send telemetry to the manager.

Moving to the OPNsense web UI:



I will enter my login credentials and head towards the *firewalls* section and then *rules* subsection.



I will then filter my firewall rules as I only need to see rules pertaining to the LAN interface currently.

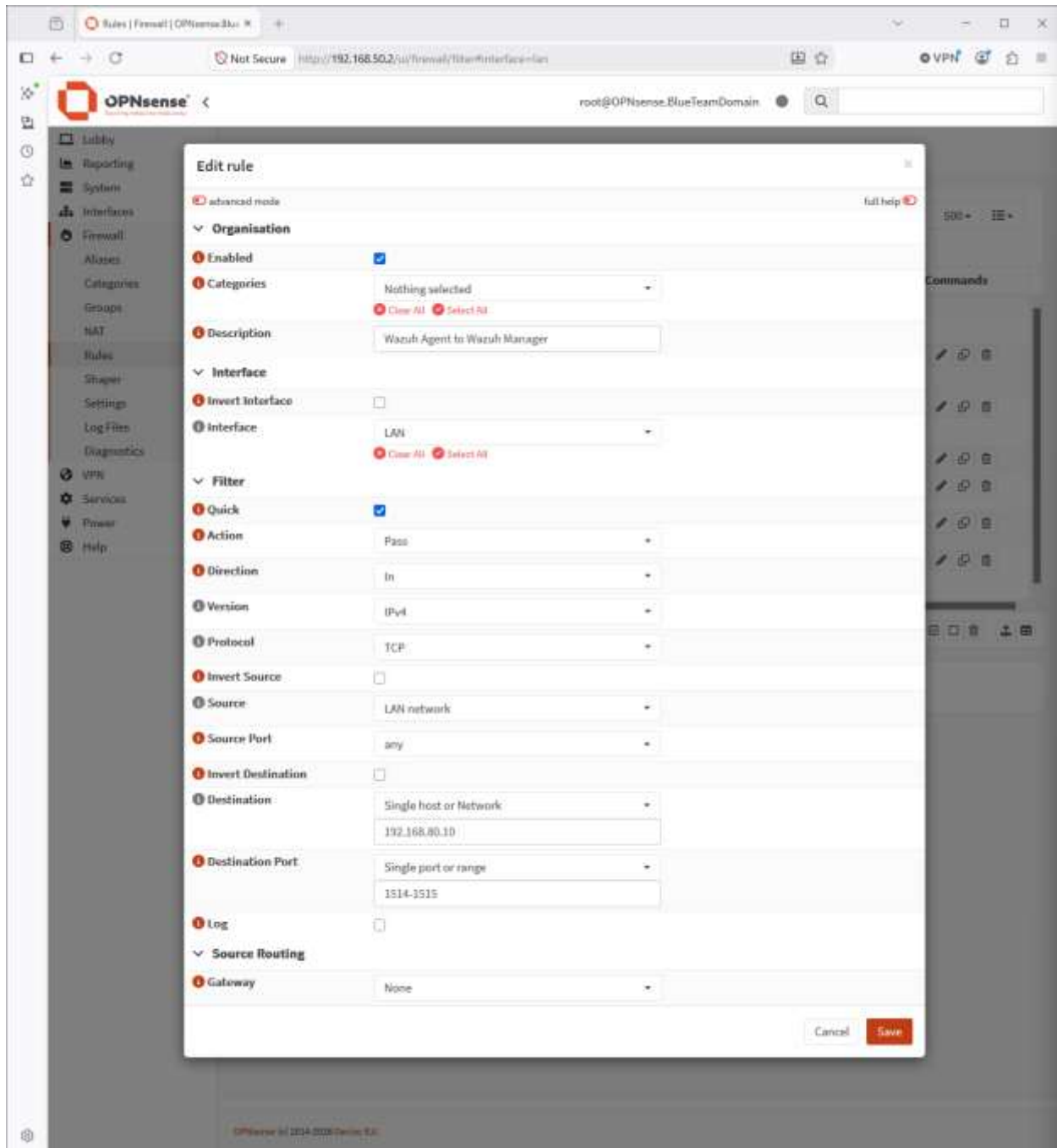
I will then create a rule that does the following:

Allow traffic into the LAN interface from any source IP from the LAN network and ports range 1514 – 1515 to the destination IP 192.168.80.10 on ports range 1514 – 1515. Telemetry being sent requires one hundred percent accuracy; thus, the protocol being used is TCP. Transmission Control Protocol ensures the telemetry will be sent to the destination since any missing packets will be retransmitted.

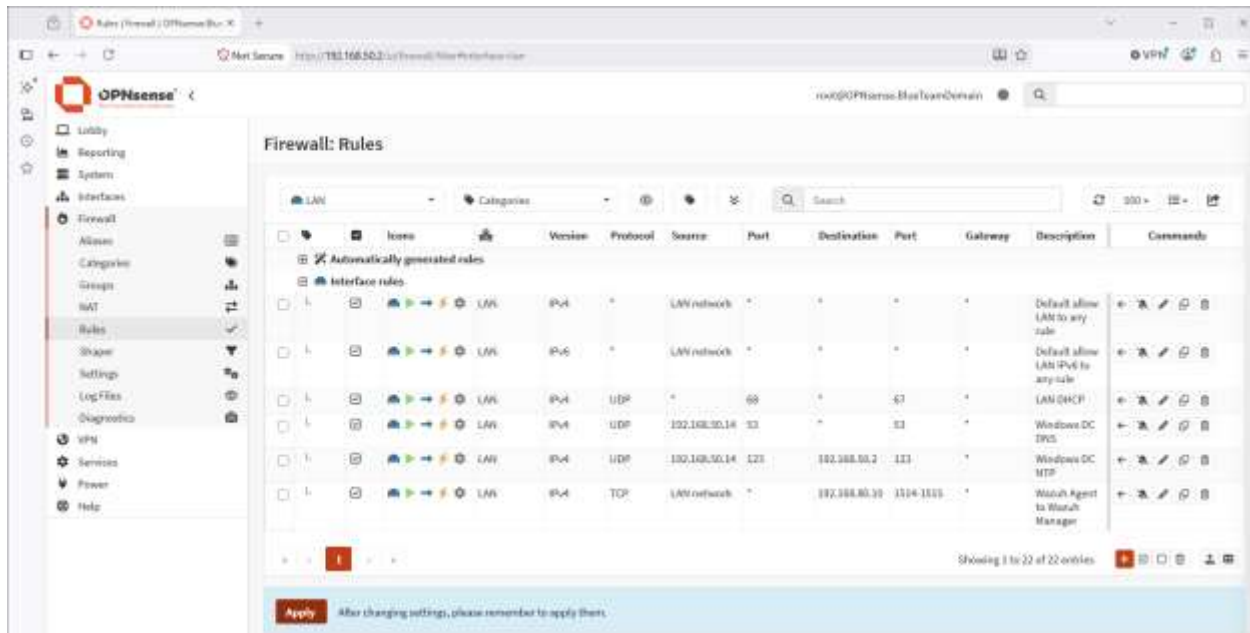
Here is an example:

Interface	Version	Direction	Protocol	Source	Port	Dest	Port	Action
LAN	IPv4	IN	TCP	LAN Network	*	192.168.80.10	1514 - 1515	Allow

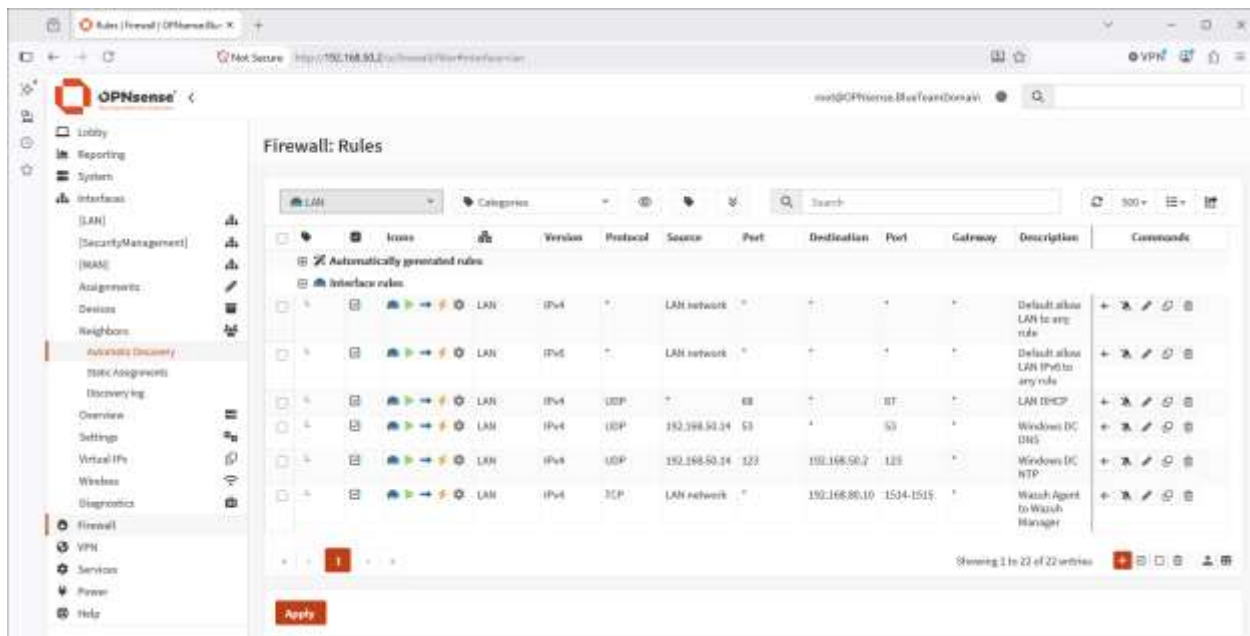
Here is a detailed image of the rule created:



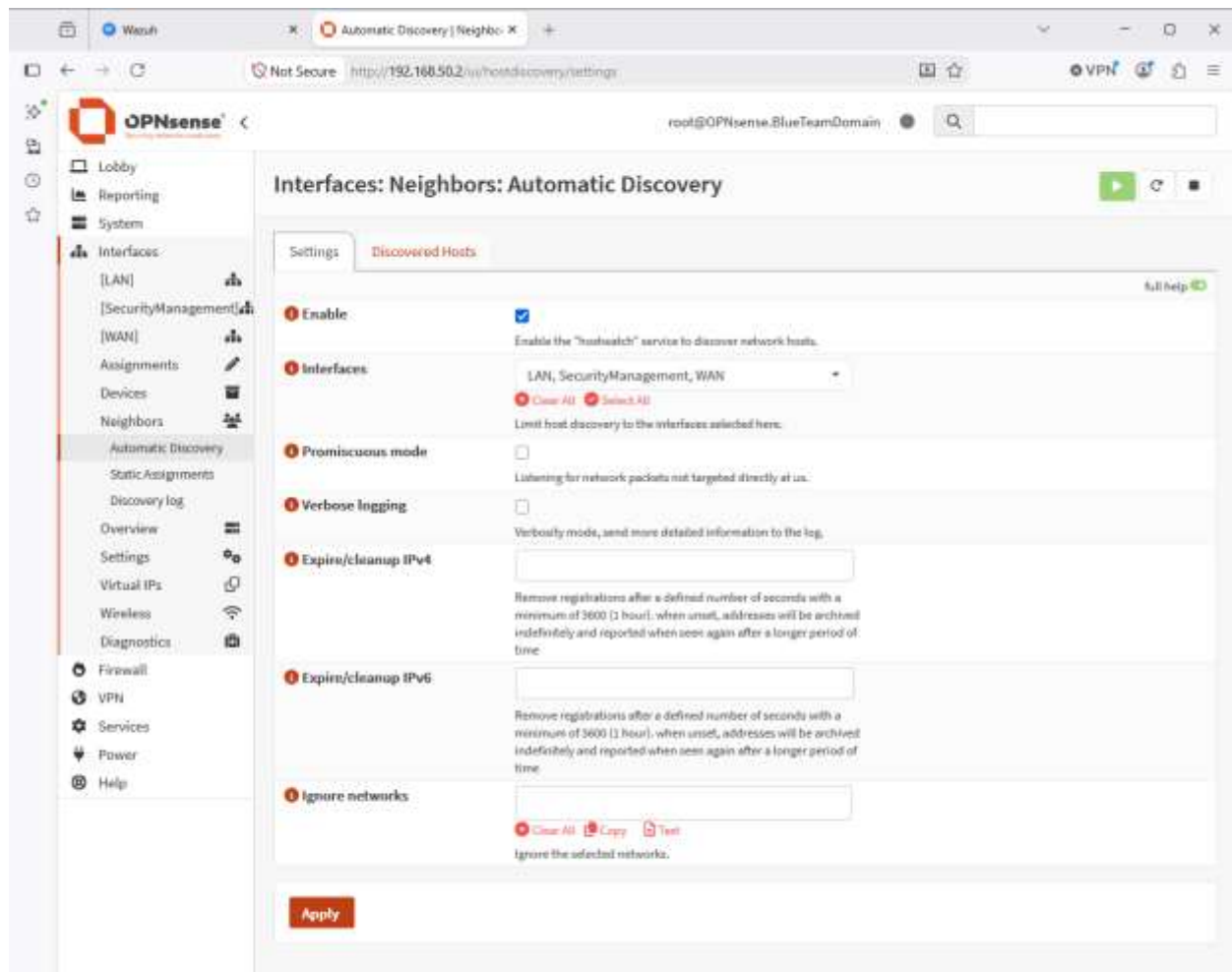
I will now apply the changes to the rule and then verify one more setting within the OPNsense web UI.



Moving on I will now head to the tab labeled **Interfaces**, select the subsection **Neighbors**, and the subsection **automatic discovery**.

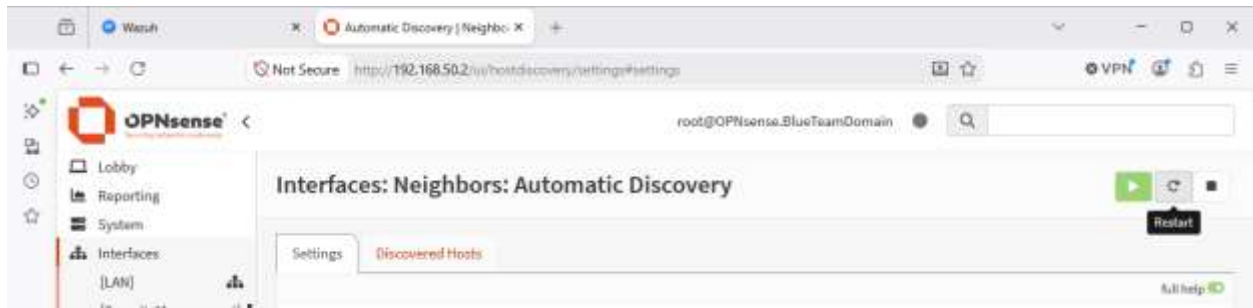


From here I will enable the setting and then in the section labeled interfaces I will select all interfaces (LAN, SecurityManagement, WAN).

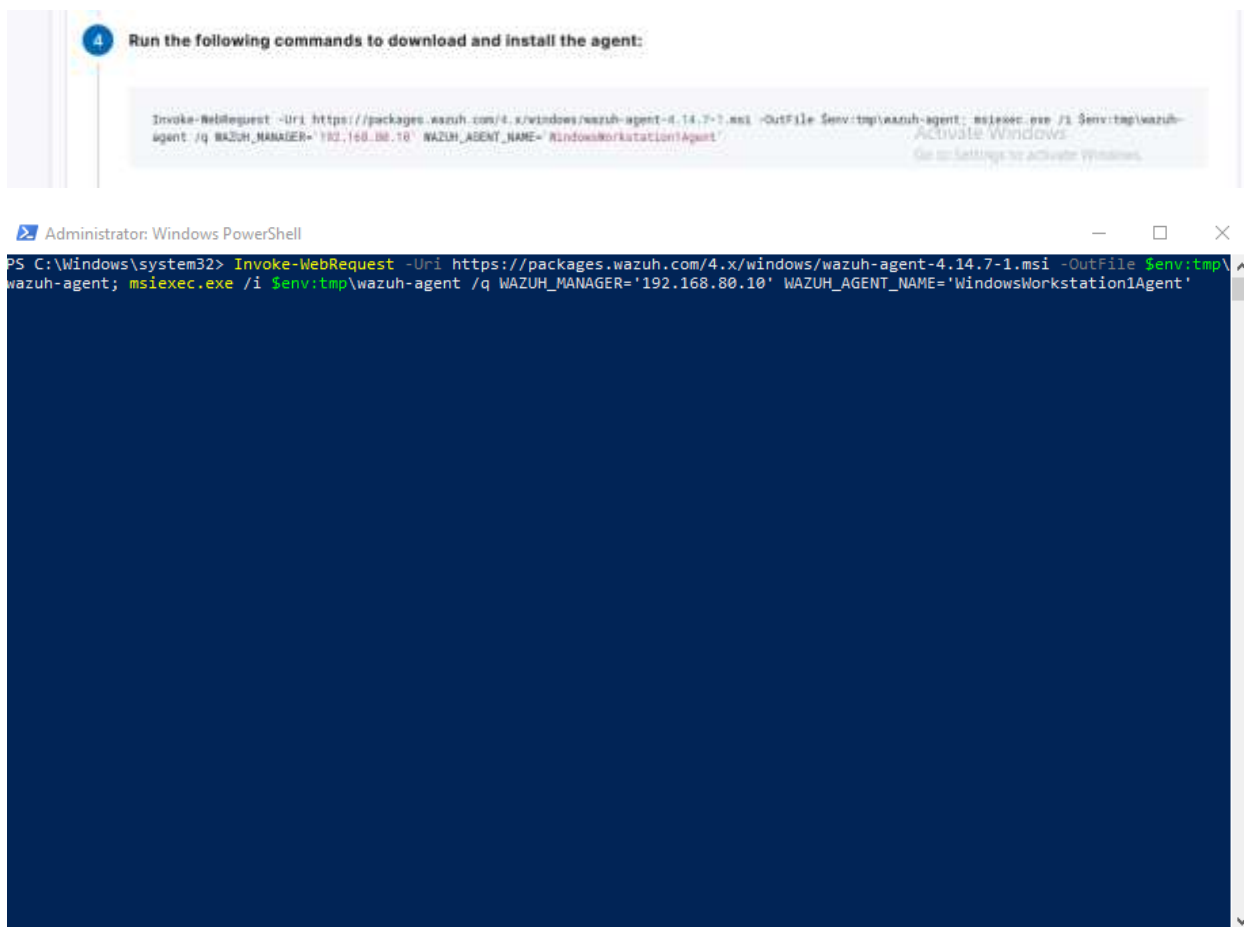


In OPNsense, the **Automatic Discovery** feature was enabled across the active interfaces (LAN, SecurityManagement, and WAN). This service actively tracks ARP and neighbor discovery data across subnets, providing administrative visibility into active endpoints regardless of IP or interface changes. Enabling cross-interface host discovery ensures that endpoints across isolated network segments remain visible within the gateway's management engine.

Once done we will restart the service as shown:



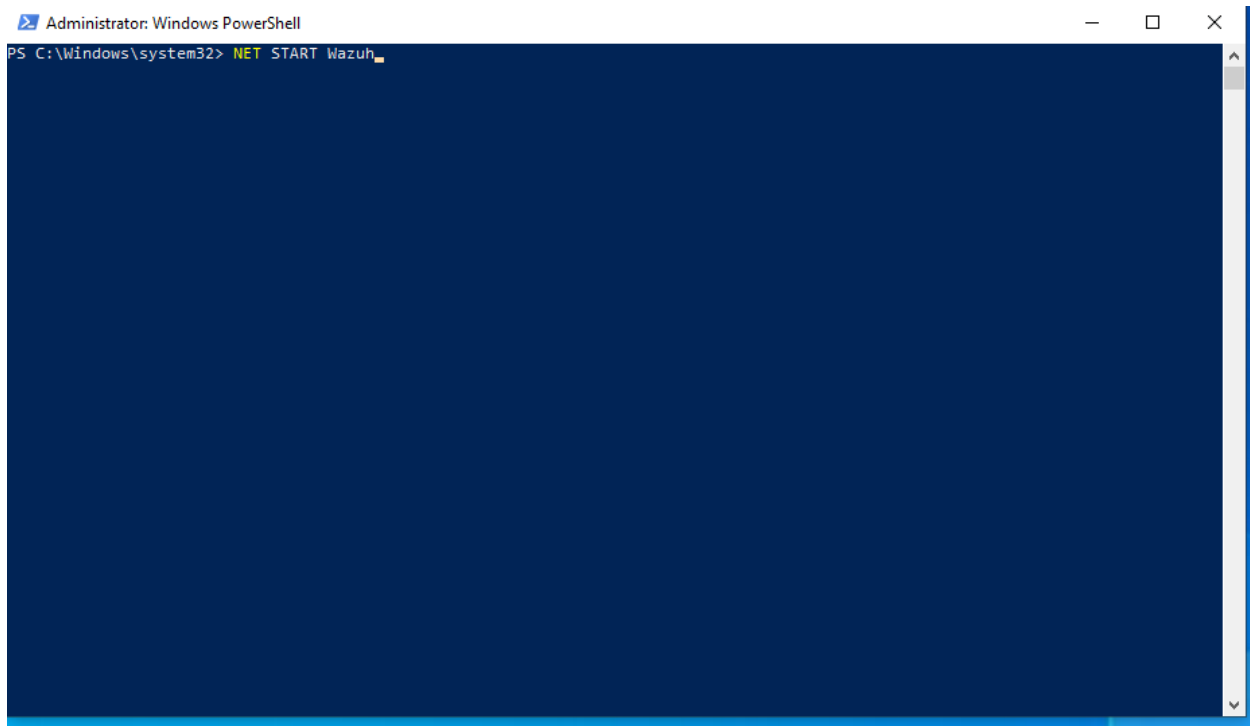
Moving on we will now copy the command generated by the Wazuh web UI and paste it into the PowerShell terminal on the Windows 10 Workstation.



After executing the command, it is now time to start the “*Wazuh*” service.

I will be using the command “*NET START Wazuh*”.

Starting services requires administrative privileges so ensure you run PowerShell as administrator.



```
Administrator: Windows PowerShell
PS C:\Windows\system32> NET START Wazuh
```

We can now verify the service is running by entering “Get-Service -Name Wazuh”, you can see the service is running.

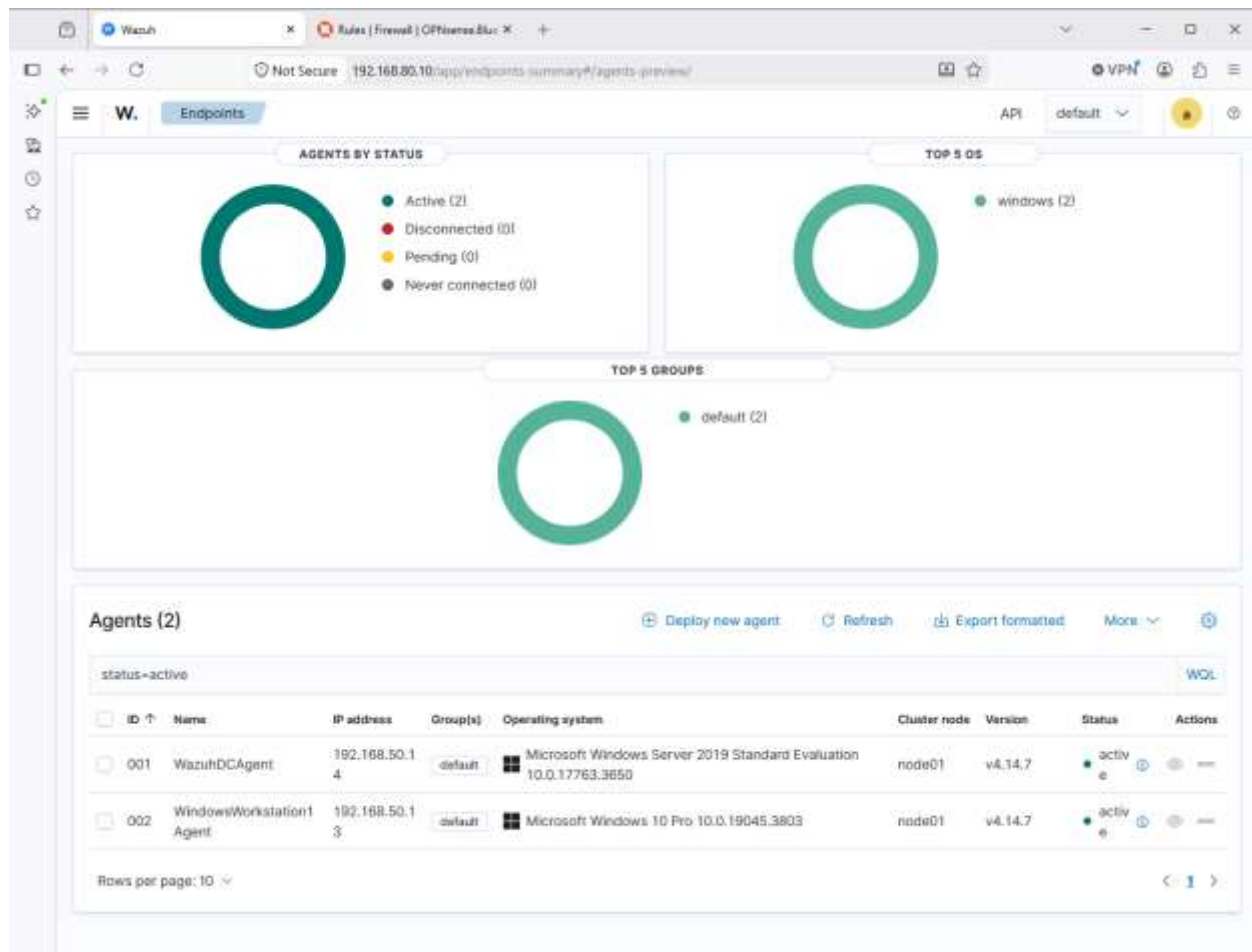


```
PS C:\Windows\system32> Get-Service -Name Wazuh

Status      Name      DisplayName
-----
Running     WazuhSvc  Wazuh

PS C:\Windows\system32>
```

Now we will switch over to the web UI for Wazuh and verify the agent is online and sending telemetry.



As you can see we successfully deployed an agent. Keep in mind that for all this to work correctly these requirements must be met:

- Firewall rule allowing traffic from between two interfaces
- Host-based Firewall on the central server must allow traffic on TCP/1514 & TCP/1515
- All information entered in the web UI to generate the command must be correct
- Ensure the service is started and running
- (Optional) Enable automatic discovery between interfaces

Validate all the requirements listed above are met and the agent should deploy without issues.

This concludes phase two of the SOC home lab project.